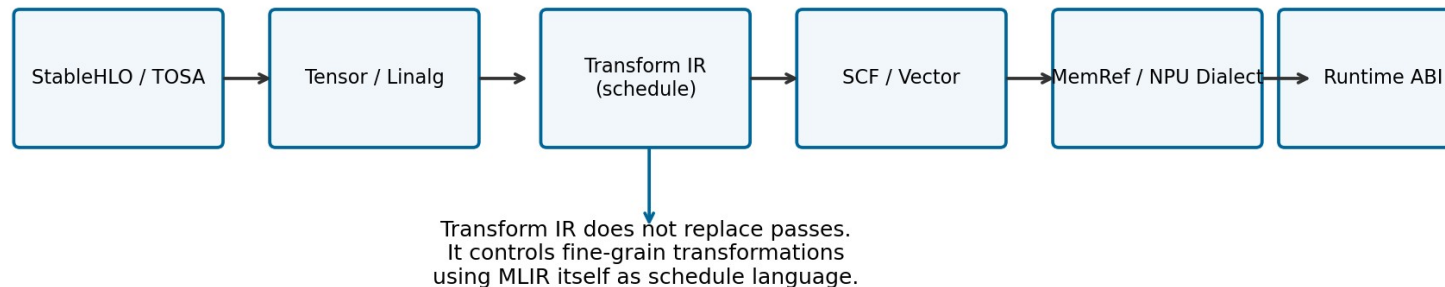


Lecture 15. Transform Dialect

tile / fuse / vectorize schedule IR for AI & NPU Compiler

Transform Dialect Position in NPU Compiler



오늘의 위치

14 강 : vector.contract -> NPU matmul mapping

15 강 : schedule itself as MLIR IR

16 강 : quantization policy and requant lowering

Learning Objectives

Transform Dialect 의 payload IR, transform IR, handle, parameter 개념을 설명한다 .

Linalg MatMul/Conv 에 대해 tile, fuse, vectorize schedule 을 IR 로 표현한다 .

schedule IR 을 NPU tile shape, SRAM footprint, accumulator type, DMA policy 와 연결한다 .

transform-interpreter 기반 실행 흐름과 regression test 전략을 설계한다 .

왜 Transform Dialect 인가 ?

NPU schedule 은 op semantics 와 HW constraints 사이의 contract 입니다 .

Pass pipeline 만으로는 특정 op/loop 에 다른 schedule 을 주기 어렵습니다 .

Transform IR 은 tile/fuse/vectorize 의사결정을 재현 가능한 IR 로 기록합니다 .

Pass vs Pattern vs Transform

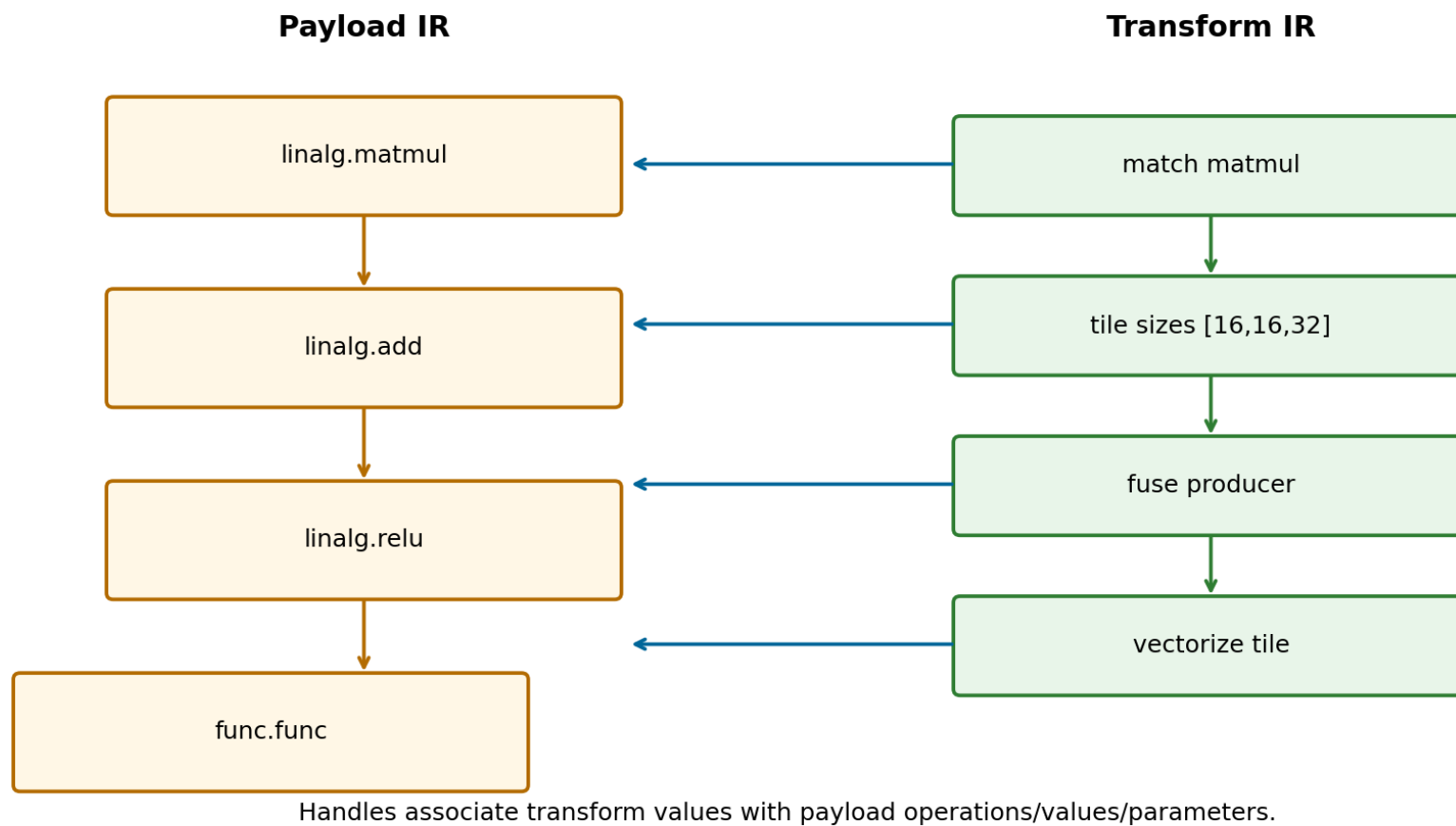
Pass: 넓은 범위에 반복 적용되는 compiler stage

Pattern: local DAG rewrite 와 canonicalization

Transform: payload IR 의 특정 op set 을 handle 로 잡아 순서 있는 변환을 지시

Payload IR vs Transform IR

Payload IR vs Transform IR



Handle / Value / Parameter

operation handle: payload op set 을 가리킴

value handle: payload SSA value set 을 가리킴

parameter: tile size, vector size 같은 attribute-time value

Execution Model

top-level transform op 에서 시작

scope payload op 에 적용

sequence body 의 transform op 를 순서대로 실행

mutation op 는 보통 입력 handle 을 consume 하고 새 handle 을 반환

Failure Modes

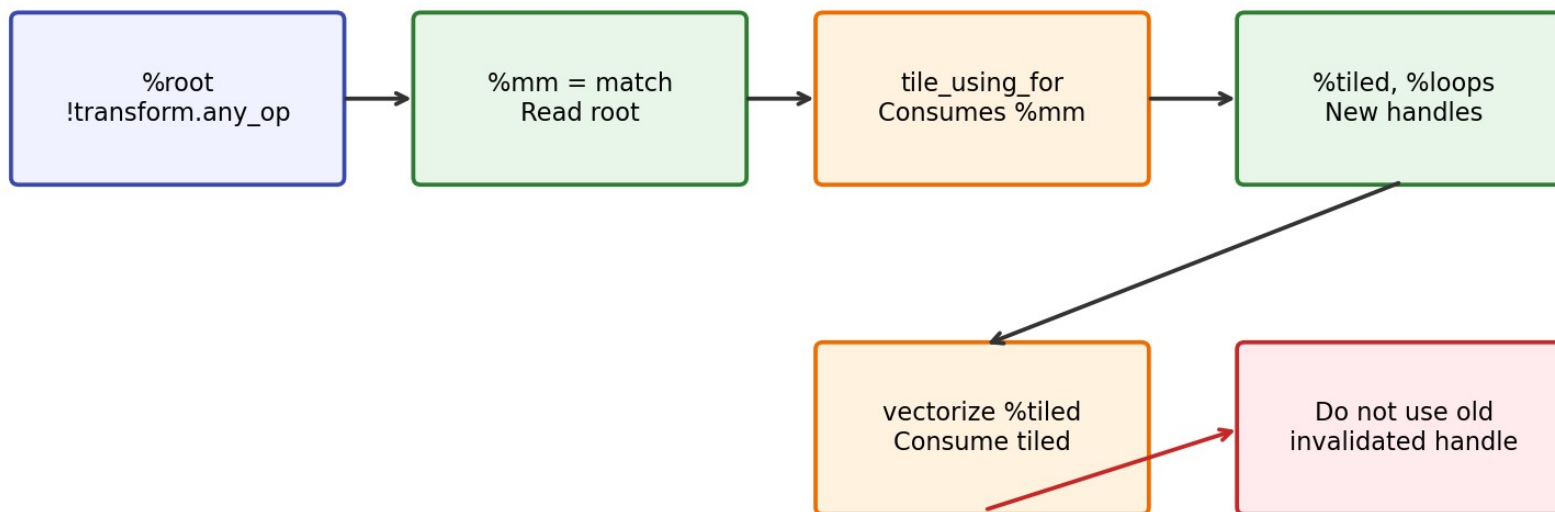
silenceable failure: precondition 실패 , fallback 가능

definite failure: postcondition/semantic violation

NPU 에서는 unsupported tile/layout/shape 를 silenceable 로 다루는 경우가 많음

Handle Invalidation

Handle Lifecycle: Read -> Consume -> Produce



NPU compiler rule: every schedule step must record which handles remain valid.

Named Sequence

transform.named_sequence 는 reusable schedule macro

entry point 는 보통 __transform_main

NPU 에서는 matmul_schedule, conv_schedule, epilogue_schedule 로 분리

Interpreter Pass

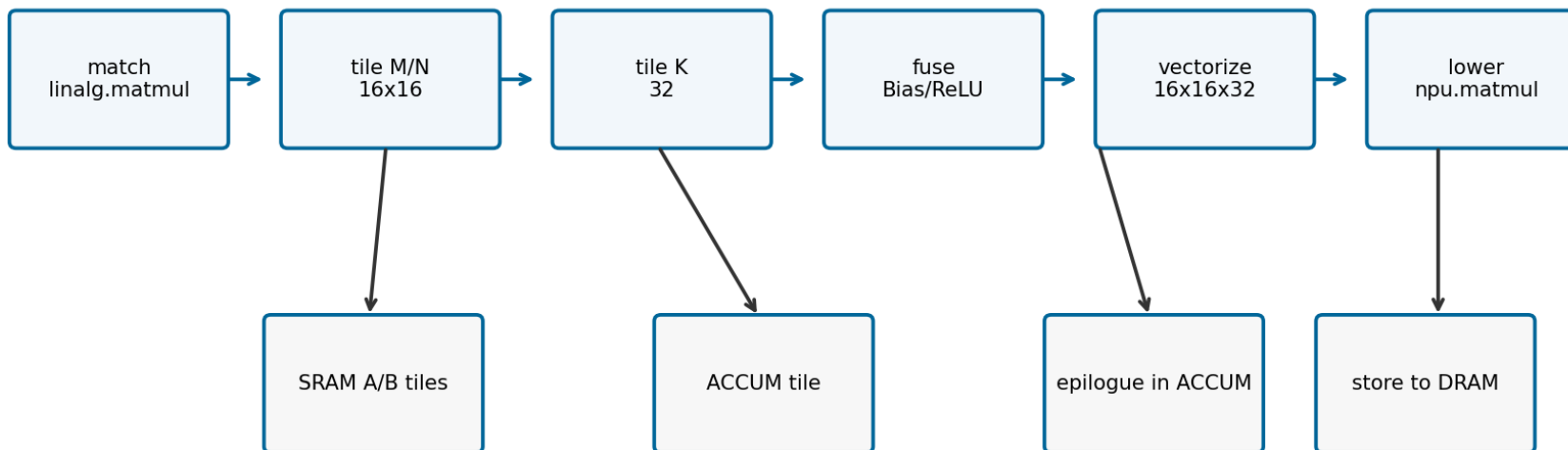
-transform-interpreter 가 named sequence 를 실행

debug payload root tag 로 적용 범위를 좁힘

CI 에서는 transform IR + FileCheck 로 schedule 회귀 테스트

NPU 관점의 Schedule IR

NPU Schedule IR for MatMul + Bias + ReLU



Schedule = hardware-facing policy: tile sizes, fusion boundary, vector shape, memory movement.

Structured Ops 와 Linalg

Linalg structured ops 는 indexing_maps 와 iterator_types 가 명확함
tile/fuse/vectorize transform 이 성숙한 영역
NPU compiler schedule 실습의 가장 좋은 시작점

Tile using for

Item	Compiler meaning	NPU meaning
target	Linalg op implementing TilingInterface	MatMul/Conv op candidate
tile sizes	static or dynamic sizes	native tile or SRAM tile
results	tiled op + generated loops	new handles for later schedule
risk	partial tiles and invalid old handles	peel/mask/fallback policy

Tile using forall

parallel tile 분배에 적합

thread/block mapping 이 있는 backend 에 유용

edge NPU 에서는 cluster/core partitioning 모델과 연결 가능

Fuse into containing op

Item	Compiler meaning	NPU meaning
producer	op feeding consumer tile	bias/requant/relu producer
container	loop/tile operation	tile-local accumulator scope
benefit	compute only needed slice	reduce DRAM traffic
risk	higher local pressure	SRAM/register pressure

Vectorize

이미 적절히 tiled 된 Linalg op 를 vector form 으로 변환
vector size 는 native tile shape 와 일치시켜야 함
partial tile 은 mask/fallback/peeling 정책 필요

Transform Pipeline Example

Pseudo schedule - check exact syntax against your MLIR revision

```
// Transform IR schedule sketch.
// Syntax is close to upstream MLIR but may need adjustment for a specific LLVM/MLIR revision.
// Check upstream Transform Dialect docs and examples/transform tests for exact current spelling.

module attributes {transform.with_named_sequence} {
  transform.named_sequence @_transform_main(%root: !transform.any_op)
    attributes {transform.readonly} {
    // 1. Select the MatMul payload operation.
    %matmul = transform.structured.match ops{["linalg.matmul"]} in %root
      : (!transform.any_op) -> !transform.any_op

    // 2. Tile M/N/K to the NPU native tile shape.
    // M=16, N=16, K=32 for INT8xINT8->INT32 accumulator.
    %tiled_matmul, %loops:3 = transform.structured.tile_using_for %matmul
      tile_sizes [16, 16, 32]
      : (!transform.any_op) -> (!transform.any_op, !transform.any_op, !transform.any_op, !transform.any_op)

    // 3. Match epilogue op and fuse it into the containing loop/tile when legal.
    %epilogue = transform.structured.match ops{["npu_graph.bias_relu"]} in %root
      : (!transform.any_op) -> !transform.any_op
    // Pseudo: exact fusion target depends on the generated loop/tile handles.
    // %fused, %new_loop = transform.structured.fuse_into_containing_op %epilogue into %loops#0
    //   : (!transform.any_op, !transform.any_op) -> (!transform.any_op, !transform.any_op)

    // 4. Vectorize the tiled MatMul. It can later be tensorized to npu_kernel.matmul.
    transform.structured.vectorize %tiled_matmul vector_sizes [16, 16, 32]
      : !transform.any_op
```

MatMul Schedule Case Study

M,N,K = 128,128,64

native tile = 16x16x32

INT8 input, INT32 accumulator

epilogue: bias + requant + relu

NPU Tile Shape Policy

compute array utilization

SRAM A/B tile footprint

ACCUM buffer size

DMA burst alignment

edge tile handling

SRAM Footprint Policy

$A_{\text{tile}} = M_{\text{tile}} \times K_{\text{tile}} \times \text{bytes}(A)$

$B_{\text{tile}} = K_{\text{tile}} \times N_{\text{tile}} \times \text{bytes}(B)$

$C_{\text{acc}} = M_{\text{tile}} \times N_{\text{tile}} \times \text{bytes}(\text{acc})$

double buffering budget 포함

Epilogue Fusion Policy

Bias/ReLU/Requant 는 accumulator locality 를 활용

LayerNorm 처럼 reduction 이 필요한 op 는 별도 schedule 검토

fusion 은 correctness + register/SRAM pressure 균형

Vectorization Policy

vector.contract 가 npu.matmul 으로 tensorize 가능한 shape 인지 확인

mask vectorization 은 hardware support 여부 필요

fallback 은 split/unroll 또는 CPU path

Bufferization Boundary

tile/fuse/vectorize 는 가능하면 tensor land 에서 수행

bufferization 후에는 memref layout/memory space 제약이 강해짐

NPU schedule 은 bufferization 전에 대부분 결정

Schedule Parameters

Schedule-as-data makes NPU policies reviewable

```
# NPU schedule spec generated from Transform Dialect design exercise.
schedule_name: matmul_bias_relu_int8
payload_match:
  op: linalg.matmul
  lhs_type: tensor<MxKxi8>
  rhs_type: tensor<KxNxi8>
  acc_type: tensor<MxNxi32>
policy:
  outer_tile: {M: 64, N: 64, K: 64}
  inner_tile: {M: 16, N: 16, K: 32}
  vector_contract: {lhs: "16x32xi8", rhs: "32x16xi8", acc: "16x16xi32"}
  epilogue_fusion: [bias, requant, relu]
  memory:
    A_tile_space: SRAM0
    B_tile_space: SRAM1
    C_acc_space: ACCUM
    double_buffer: true
  fallback:
    unsupported_layout: repack_or_cpu
    k_remainder: split_or_mask
    dynamic_shape: runtime_tile_selection
checks:
  max_sram_bytes_per_core: 262144
  max_accumulator_bytes: 65536
  require_dma_alignment_bytes: 64
```

Matching Strategy

operation name match: linalg.matmul, linalg.generic

attribute/type filtering: dtype, layout, quant

payload root scope 를 좁혀 unintended rewrite 방지

PDL / foreach_match 개념

PDL 로 complex match 를 표현 가능

foreach_match 는 matcher/action pair 를 순회 적용

NPU 에서는 pattern 별 schedule library 설계에 유용

Debugging Recipe

transform.print 로 handle 확인

-transform-dialect-check-uses 로 use-after-free 탐지

small payload IR 부터 FileCheck 작성

negative test: unsupported shape/layout

Regression Test Matrix

Transform Dialect Regression Matrix

Schedule	Expected IR	Legality	Perf Proxy
tile sizes tile sizes	CHECK	CHECK	CHECK
fusion fusion	CHECK	CHECK	CHECK
vectorization vectorization	CHECK	CHECK	CHECK
failure path failure path	CHECK	CHECK	CHECK

For NPU: schedule correctness is not enough; check memory footprint, vector shape, and fallback cases.

Failure and Fallback

shape 가 native tile 과 다르면 peel/mask/split 선택
layout mismatch 는 transpose/repack 또는 fallback
schedule failure 는 diagnostic 이 설명 가능해야 함

Custom NPU Transform Ops

Out-of-tree extension sketch

```
// Pseudo C++ skeleton for an out-of-tree NPU Transform Dialect extension.
// This is a teaching artifact, not a drop-in build file.

#include "mlir/Dialect/Transform/IR/TransformDialect.h"
#include "mlir/Dialect/Transform/Interfaces/TransformInterfaces.h"

namespace npu {
class NPUTransformDialectExtension
    : public mlir::transform::TransformDialectExtension<NPUTransformDialectExtension> {
public:
    NPUTransformDialectExtension() {
        // declareGeneratedDialect<npu::NPUKernelDialect>();
        // registerTransformOps<ScheduleMatmulTileOp, PromoteToSramOp, TensorizeToNpuOp>();
    }
};
}

// Design sketch:
// npu.schedule.matmul_tile %target {m=16, n=16, k=32}
// npu.schedule.promote_to_sram %tile {space="SRAM0", double_buffer=true}
// npu.schedule.tensorize_to_npu %vector_contract {instruction="mma_i8_i32"}
```


실습 A - Handle Annotation

payload IR 에서 linalg.matmul/add/relu 를 찾기

각 transform handle 이 어떤 payload op set 을 가리키는지 표 작성

consume 이후 사용 금지 handle 표시

실습 B - Tile/Fuse/Vectorize

MatMul schedule transform IR 작성

tile size 16x16x32 적용

bias/relu fusion 위치 결정

vector size 와 npu tile shape 매핑

실습 C - NPU Schedule Spec

schedule parameter table

SRAM footprint table

fallback policy

FileCheck regression plan

Common Mistakes

old handle 을 계속 사용

tile size 만 보고 memory footprint 를 무시

vectorize 를 너무 일찍 적용

schedule IR 과 pass pipeline 책임 혼동

Homework

MatMul+Bias+ReLU schedule IR 완성

Conv2D NHWC/HWCF schedule variant 작성

unsupported K remainder case negative test 작성

Takeaways

Transform Dialect 는 schedule 을 IR 로 만드는 도구

NPU compiler 에서는 tile/fuse/vectorize policy 를 재현 가능하게 관리

handle lifetime 과 failure mode 를 테스트해야 함

References

MLIR Transform Dialect: <https://mlir.llvm.org/docs/Dialects/Transform/>

MLIR Transform Dialect Tutorial: <https://mlir.llvm.org/docs/Tutorials/transform/>

Transform Tutorial Chapter 1: <https://mlir.llvm.org/docs/Tutorials/transform/Ch1/>

Transform Tutorial Chapter H - Halide Schedule: <https://mlir.llvm.org/docs/Tutorials/transform/ChH/>

MLIR Linalg Dialect: <https://mlir.llvm.org/docs/Dialects/Linalg/>